

User Mode vs Supervisor Mode (Kernel Mode)

1 What is CPU Operating Mode?

Modern CPUs support multiple operating modes to enhance **security**, **efficiency**, and **control**. The two primary modes are:

- **User Mode** – Runs user applications with limited privileges.
- **Supervisor Mode (Kernel Mode)** – Runs the OS with full privileges.

2 Key Differences Between Modes

Feature	User Mode	Supervisor Mode (Kernel)
Hardware Access	Not allowed	Full access to hardware
Instruction Set	Limited (non-privileged)	Full (including privileged instructions)
Memory Access	Restricted (sandboxed)	Complete control
Responsible Software	User apps (e.g., browser, games)	Operating system kernel
Risk to System	Low (safe)	High (if errors occur)
System Calls Allowed?	No	Yes

Table 1: Comparison of User Mode vs Supervisor Mode

3 Why Separation is Important

- **Security:** Keeps user applications from directly accessing or damaging system components.
- **Stability:** One faulty app cannot crash the entire OS.
- **System Calls:** Applications must request OS services through controlled interfaces.

4 Switching Between Modes

4.1 How it Works

1. A user application requests a service (e.g., open file).
2. A **System Call** is triggered.
3. CPU switches from **User Mode** to **Supervisor Mode**.
4. OS handles the request and switches back.

Application (User Mode) $\Rightarrow$ System Call $\Rightarrow$ Kernel (Supervisor Mode)

Code Example (C)

```
#include <fcntl.h>
int fd = open("data.txt", O_RDONLY); // Triggers a system call
```

Listing 1: System Call to Open a File

5 Real-World Analogy

- **User Mode:** Hotel guest – can use room and services but can't access the back office.
- **Supervisor Mode:** Hotel manager – full access including restricted areas.

6 Key Takeaways

Summary

- **User Mode** is safe, limited, and used by applications.
- **Supervisor Mode** is powerful, dangerous if misused, and used by the OS.
- Mode switching enables controlled access to hardware and OS services.
- Always use system calls or APIs to interact with hardware from user space.